

CURL API Testing Tutorial: How to Use CURL to Send API Requests

1. Introduction to cURL

cURL (Client URL) is a command-line tool used to transfer data to or from a server using various protocols such as HTTP, HTTPS, FTP, and more. It is widely used for **API testing**, debugging backend services, and automating HTTP requests without using graphical tools like Postman.

In real development environments, cURL is often the fastest way to verify whether an API endpoint is working correctly because it removes UI complexity and exposes the raw request/response behavior.

2. Why Use cURL for API Testing?

Key practical reasons developers rely on cURL:

- Lightweight and preinstalled on most systems (Linux, macOS)
- Works directly from the terminal or command prompt
- Useful for automation scripts and CI/CD pipelines
- Allows precise control over headers, methods, and request body
- Helps debug APIs at the protocol level

Hard reality: If you work with backend systems, containers, or production debugging, you will eventually need cURL. GUI tools are convenient, but cURL is the baseline skill.

3. Basic cURL Command Syntax

```
curl [options] [URL]
```

Example:

```
curl https://api.example.com/users
```

This sends a **GET request** to the specified URL.

4. HTTP Methods Used in API Testing

The most common HTTP methods tested using cURL are:

Method	Purpose
GET	Retrieve data
POST	Send new data
PUT	Update existing data
PATCH	Partially update data
DELETE	Remove data

5. Sending a GET Request

Example

```
curl https://jsonplaceholder.typicode.com/posts
```

With Query Parameters

```
curl "https://api.example.com/users?id=10"
```

With Headers

```
curl -H "Authorization: Bearer YOUR_TOKEN"  
https://api.example.com/profile
```

6. Sending a POST Request

POST is used to create new resources.

Basic POST Request

```
curl -X POST https://api.example.com/users
```

POST with JSON Data

```
curl -X POST https://api.example.com/users
-H "Content-Type: application/json"
-d '{
    "name": "John",
    "email": "john@example.com"
}'
```

Important flags:

- **-X**: Specifies HTTP method
 - **-H**: Adds headers
 - **-d**: Sends request body data
-

7. Sending a PUT Request

Used to update an existing resource.

```
curl -X PUT https://api.example.com/users/1
-H "Content-Type: application/json"
-d '{
    "name": "Updated Name"
}'
```

8. Sending a PATCH Request

Used for partial updates.

```
curl -X PATCH https://api.example.com/users/1
-H "Content-Type: application/json"
-d '{
    "email": "new@email.com"
}'
```

9. Sending a DELETE Request

```
curl -X DELETE https://api.example.com/users/1
```

10. Adding Headers to Requests

Headers are critical for authentication and content negotiation.

```
curl -H "Authorization: Bearer TOKEN"  
-H "Content-Type: application/json"  
https://api.example.com/data
```

Common headers:

Header	Purpose
Authorization	Authentication token
Content-Type	Data format (JSON, XML)
Accept	Expected response format

11. Sending Form Data

```
curl -X POST https://api.example.com/login  
-d "username=admin"  
-d "password=1234"
```

12. Uploading Files Using cURL

```
curl -X POST https://api.example.com/upload  
-F "file=@document.pdf"
```

Flag:

- **-F** : Used for file upload (multipart/form-data)

13. Saving Response to a File

```
curl -o response.json https://api.example.com/data
```

14. Viewing Response Headers

```
curl -I https://api.example.com
```

15. Verbose Mode (Debugging)

```
curl -v https://api.example.com
```

This shows:

- Request headers
- Response headers
- Connection details

If something fails in production, this flag is often the first diagnostic step.

16. Authentication Methods

Basic Authentication

```
curl -u username:password https://api.example.com
```

Bearer Token Authentication

```
curl -H "Authorization: Bearer YOUR_TOKEN"  
https://api.example.com
```

17. Realistic API Testing Workflow (What Actually Happens in Practice)

Typical sequence:

1. Check endpoint using GET
2. Add authentication header
3. Send POST request with JSON body
4. Verify response status code
5. Debug using verbose mode
6. Automate using scripts

If you skip step-by-step validation, you will waste time debugging symptoms instead of root causes.

18. Common cURL Options Cheat Sheet

Option	Meaning
-X	HTTP method
-H	Header
-d	Request body
-o	Save output to file
-I	Show headers only
-v	Verbose mode
-u	Username and password
-F	File upload

19. Typical Mistakes Beginners Make

1. Forgetting Content-Type header
2. Using wrong HTTP method
3. Sending invalid JSON
4. Missing authentication token
5. Ignoring response status codes

These are not theory problems — they are the real reasons APIs fail in practice.

20. Conclusion

cURL is not just another tool — it is the lowest-level practical interface for API communication. If you can test an API reliably with cURL, you understand how HTTP actually works. If you rely only on GUI tools, you understand the interface, not the protocol.

For backend development, DevOps, or API debugging, cURL competence is a baseline technical skill, not an optional one.